Toward Real-Time Marine Vessel Monitoring on AWS

Gopi Krishnan
Student ID: X25112627
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25112627@student.ncirl.ie

***Abstract*—Cruise vessels increasingly carry dense sensor arrays for engine, fuel, ballast, and hull condition monitoring, but a fixed inspection schedule still leaves the bridge crew blind between checks. This paper presents the design, implementation, and live cloud deployment of a fog-to-serverless pipeline that closes that gap: two simulated vessels, five sensor types each, feed a Tornado-based fog node that windows, aggregates, and threshold-evaluates every reading stream before dispatching a batched message downstream. The backend runs entirely on managed AWS services: an Amazon SQS queue decouples ingestion from processing, an AWS Lambda function writes aggregated windows to DynamoDB, and a second Lambda function answers the dashboard's REST API behind an Amazon API Gateway REST API through a flat dict-based route table rather than a class-based web framework. Its static dashboard assets sit in a separate Amazon S3 bucket, entirely detached from the API tier. The complete system was deployed to a real AWS account, where a DynamoDB Scan-pagination bug and a missing message-batching capability were found and fixed before deployment, and a third defect—every static asset on the deployed dashboard 404ing—was found only once the live page was opened in an actual browser, not merely queried through its API. What follows lays out that architecture, the paths not taken and why, and the proof gathered by putting the running system itself to the test.**

***Keywords*—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, maritime sensing**

I. Introduction

A bridge crew relying on a fixed rounds schedule finds out about a fuel burn spike, a ballast imbalance, or rising hull vibration only at the next scheduled check, not when it starts. Continuous sensing changes that trade-off: predictive maintenance research applying machine learning to time-series sensor streams from marine engines has shown that condition data collected continuously, not periodically, is what makes early warning possible in the first place [1].

The National Institute of Standards and Technology defines cloud computing around five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service [2]. Fog computing extends that model to the edge of the network, described by Bonomi et al. as a platform distinguished by low latency, wide geographic distribution, and support for a large number of nodes [3]. This project follows that definition directly: windowed aggregation and threshold evaluation happen on a fog node co-located with the sensors, and only the resulting aggregate-plus-alerts payload, not every raw reading, crosses the network boundary into the cloud.

Maritime deployments sharpen that trade-off instead of merely illustrating it: a vessel's link back to shore is frequently narrower or less reliable than a fixed building's network connection, so aggregating and evaluating readings on board, before anything leaves the vessel, is not only a latency optimisation but a bandwidth constraint the fog layer absorbs on the vessel's behalf.

This paper documents a fog and edge computing pipeline for marine vessel condition monitoring, built to satisfy three requirements. First, an edge layer spanning five distinct sensor types across two vessels, each stream sampled and dispatched on its own configurable schedule, feeding a fog node that buffers the readings into rolling windows, computes aggregates, checks them against alert thresholds, and forwards the result onward. Second, a scalable backend layer, built from managed queue and function-as-a-service mechanisms, that processes the fog node's output and exposes a responsive dashboard comparing both vessels side by side. Third, deployment and testing of that backend on a public cloud platform, verified against the deployed, running system instead of assumed correct from local tests alone.

The remainder of this paper is organised as follows. Section II describes the system's architecture and the design decisions taken at each layer, including a critical comparison against alternatives that were not chosen and the security posture of the resulting deployment. Section III describes the concrete implementation: software components and libraries, the automated test suite, continuous integration, and the AWS deployment together with the defects it surfaced. Section IV evaluates the deployed system against measured evidence. Section V concludes with a critical reflection on the project and directions for future work.

II. System Architecture and Design

A. Sensor and Fog Layer

The simulated deployment models two cruise vessels, `vessel-a` and `vessel-b`, each carrying five sensor types: engine room temperature (`engine_room_temp_c`),

TABLE I
ALERT THRESHOLDS (EVALUATED ON THE WINDOW AGGREGATE)

Sensor Type	Rule	Alert Key
engine_room_temp_c	avg > 75	engine_overheat_risk
fuel_consumption_lph	avg > 350	fuel_burn_excessive
ballast_water_level_pct	avg > 90	ballast_overfill_risk
hull_vibration_mm	max > 15	hull_stress_warning

fuel consumption (fuel_consumption_lph), ballast water level (ballast_water_level_pct), hull vibration (hull_vibration_mm), and passenger count. Each sensor process reads its cadence from a pair of environment variables, SAMPLE_INTERVAL and DISPATCH_INTERVAL, kept as two genuinely separate settings rather than one value standing in for both, so how often a reading is taken and how often it is sent can be tuned independently of each other. Each process follows a bounded random walk clamped to a physically plausible range—for example engine room temperature steps by up to 4.0 degC across a 20-90 degC range, hull vibration by up to 1.5 mm/s across 0-20 mm/s—so the trace drifts smoothly from one sample to the next instead of jumping to an unrelated figure on every tick.

The two vessels are not configured identically. For a given sensor type the compose file staggers vessel-a's and vessel-b's DISPATCH_INTERVAL instead of sharing one value: engine_room_temp_c dispatches every 8 seconds for vessel-a and every 9 seconds for vessel-b, while hull_vibration_mm dispatches every 14 seconds for vessel-a and every 10 seconds for vessel-b. The offset is deliberate: with both vessels dispatching on an identical cadence, most windows across the fleet would tend to close in the same flush cycle, which is exactly the collision case the composite window_end#site_id sort key in Section II-B is built to survive, so staggering the intervals keeps that code path exercised in practice instead of left untested by coincidence.

The fog node is a Tornado HTTP service that ingests batches over its /ingest endpoint and buffers them per (sensor_type, site_id) key in plain module-level dictionaries, with no lock, queue, or double-buffer object protecting them at all. Correctness rests entirely on Tornado's IOLoop being single-threaded and non-preemptive between await points: the ingest handler never awaits mid-mutation, and the periodic flush callback never awaits mid-snapshot, so the two can never interleave partway through a write. Regardless of how full the buffer is when the interval fires, the callback swaps in an empty dictionary and hands the captured window off for processing. For each key it works out the minimum, maximum, mean, latest reading, and sample count, checks the result against the alert rules in Table I, and once the whole window has been handled, publish_batch() forwards the readings on to Amazon SQS, chunking them into groups of at most ten entries per SendMessageBatch call—a design comparable to the windowed stream aggregation approach fog-tier platforms take when reducing raw IoT streams before they leave the edge [4].

Table I's rules are not stored as strings later matched through an if/elif chain: each rule's op field holds operator.gt itself, called against the summary field and the limit inside evaluate(), so changing a threshold means editing one entry in RULES and nothing else. The /thresholds endpoint the dashboard reads to draw its threshold lines is built from that same RULES list on every call instead of a separately maintained payload, so the two can never drift apart. One further detail worth stating plainly: aggregate()'s latest field takes the last reading in arrival order within the window, not the reading with the highest timestamp—since sensors dispatch each batch in order, arrival order already matches chronological order, so latest is a plain list-index lookup instead of a timestamp comparison across the whole batch.

B. Scalable Backend Layer

The backend layer is built entirely from managed, scalable AWS primitives. An Amazon SQS standard queue, mvs-vessel-agg, sits between the fog node's dispatch rate and the backend's processing rate, applying the queue-based decoupling pattern that a recent independent benchmark of four production message brokers found central to their scalability characteristics under load [5]. Messages arriving on that queue automatically invoke an AWS Lambda function, mvs-processor, via an event-source mapping, and each invocation writes one aggregated window into the mvs-readings DynamoDB table, keyed by sensor_type as the partition key and a composite window_end#site_id sort key—the same narrow, well-known key-based lookup approach behind Amazon's original key-value store design [6]. That composite key is not incidental: with window_end alone as the sort key, an engine_room_temp_c aggregate from vessel-a and one from vessel-b closing in the same flush cycle would collide on an identical primary key, and whichever write landed second would silently overwrite the first. Appending the site identifier keeps the two vessels' concurrent writes apart without needing a secondary index.

A commonly raised critique of Lambda-based architectures is cold-start latency: an invocation on a function with no currently warm execution environment incurs an initialisation delay before the handler runs, a cost documented empirically across a large commercial serverless deployment [7]. This is judged acceptable here for two reasons specific to this workload. First, nothing waits on the fog-to-processor hop in real time—SQS delivers the batch whenever the function next spins up, so an occasional initialisation delay just shifts when a reading lands in DynamoDB by a moment, and no browser tab is sitting there waiting on that particular call to return. Second, the dashboard-facing Lambda is invoked frequently enough by the live dashboard's 2.5-second polling that its execution environment rarely goes cold during an active demonstration session.

The dashboard's query layer builds every view from two primitives. recent_windows(sensor_type, limit) issues one DynamoDB Query against the sensor_type

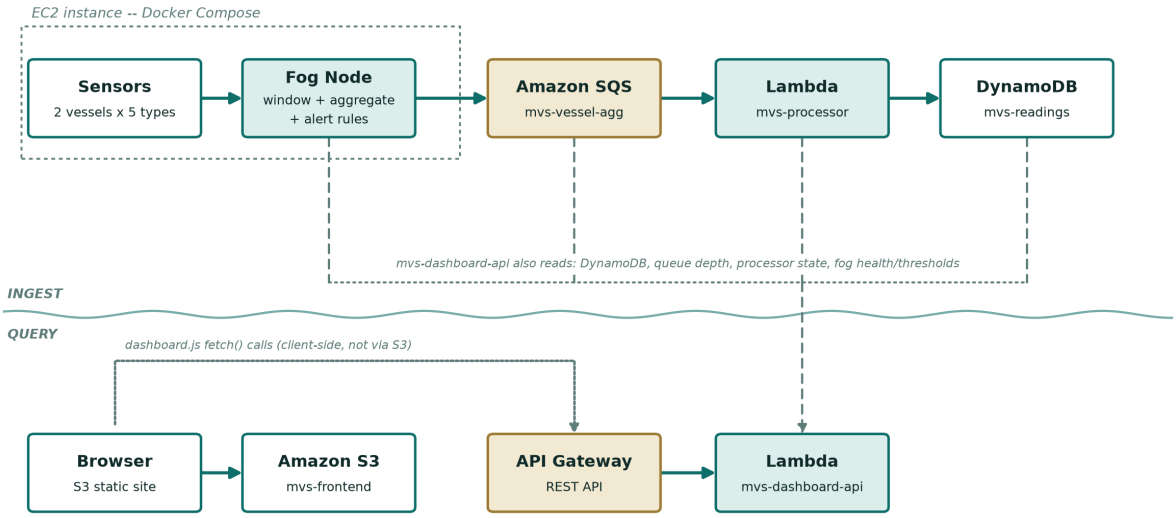

Fig. 1. Deployment topology: an ingest lane (sensors through DynamoDB) and a query lane (browser through the dashboard API) either side of a waterline; the backend is fully serverless except the EC2-hosted sensor and fog tier.

partition key with `ScanIndexForward=False`, so DynamoDB returns the newest windows first without a client-side sort, then reverses that list to oldest-first before returning it, since every chart and table consumer renders left to right chronologically. `latest_by_site(sensor_type)` calls `recent_windows` and walks the oldest-first result, letting each `site_id`'s entry overwrite the previous one in a dict, so whichever row is seen last for a given vessel is, by construction, that vessel's newest window—no separate `max()` or timestamp sort is needed. `vessel_report()`, which feeds the dashboard's Bridge Console panel, calls `latest_by_site` once per sensor type and reshapes the result into a two-column `vessel-a`/`vessel-b` comparison instead of a per-vessel card grid, matching the side-by-side comparison Section I's requirements called for.

C. A Flat Dispatch Table for the Dashboard API

The dashboard-facing Lambda, `mvs-dashboard-api`, sits behind a real Amazon API Gateway REST API and answers every `/api/*` route the dashboard calls: readings, vessels, voyage-log, thresholds, health, backend-stats. Every one of those routes is a fixed, static path with no dynamic path segment, so `mvs-dashboard-api` routes them through a flat dict keyed by (`method`, `path`)—a plain hash-map lookup rather than an ordered pattern-matching table scanned linearly or a character-by-character trie walked segment by segment, both approaches better suited to APIs that do have path parameters to extract. Reusing the local Tornado dashboard's URL-routing mechanism inside the Lambda was considered and set aside: Tornado's router depends on its IOLoop and RequestHandler request/response cycle, neither of which exists inside a single synchronous Lambda invocation, so reusing it directly would have meant either an ASGI/WSGI compatibility shim or reimplementing request dispatch by hand regardless—at which point a flat dict is the simpler, more auditable choice for six fixed routes, following the API Gateway design pattern of centralising routing logic behind one gateway layer the client only ever talks to directly [8].

`mvs-dashboard-api` answers requests from four upstream sources: the DynamoDB table for readings and vessel comparisons, the SQS queue's attributes for backlog depth, `mvs-processor`'s Lambda metadata to report whether the ingestion function is active, and the fog node's `/health` and `/thresholds` endpoints over the Elastic IP the EC2 instance is bound to. This last dependency carries a known constraint: a raw EC2 public IP address on a non-standard port served over plain HTTP is exactly the traffic shape that campus and corporate WiFi networks commonly block by policy, so hosting the dashboard itself on EC2 would have reintroduced a reachability risk the fully serverless path avoids. The dashboard's static assets are instead served directly from an Amazon S3 bucket, and `dashboard.js`'s `API_BASE` constant, a `%%API_BASE%%` token substituted with the actual API Gateway URL at S3-deploy time, tells the browser-side JavaScript where to send its requests—left as the literal token for local development, where the same Tornado process serves both the API and the static files from one origin.

D. Deployment Topology

Fig. 1 summarises the resulting topology as two lanes either side of a waterline: the ingest path above, where sensors, the fog node, SQS, `mvs-processor`, and DynamoDB run left to right, and the query path below, where the browser reaches the static frontend on S3 and calls the live API directly through API Gateway. Sensor processes and the fog node run in Docker containers on an EC2 instance with no key pair ever issued for it, reachable only via AWS Systems Manager Session Manager, and with its security group left closed on every port except the fog node's port 8000. `mvs-dashboard-api` reads from DynamoDB and from the three further sources

described above to answer the dashboard's REST API calls, while the dashboard's static frontend is served independently from S3, reached directly by the browser. Strip the sensor and fog tier out of this picture and everything remaining sits behind a managed AWS service, not a server this project runs and patches itself.

E. Critical Analysis of Alternative Architectures

The sections below examine each major design decision individually, noting concretely why an alternative was ruled out or, where an unconventional choice was kept, exactly what constraint justifies it. The fog node's lock-free buffering was itself a deliberate departure from the `threading.Lock` and `asyncio.Lock` approaches common in equivalent systems: plain dictionaries with no synchronisation primitive at all are only correct because Tornado's IOLoop guarantees the ingest handler and the periodic flush callback never run concurrently mid-mutation. This would be a genuine data race under a multi-threaded framework, so the simplification is bound tightly to Tornado's specific concurrency model, not a general pattern this project claims is always safe.

Apache Kafka and RabbitMQ were considered in place of SQS for the fog-to-processor link. An independent benchmark measuring throughput and latency across Kafka, RabbitMQ, ActiveMQ Artemis, and Redis found meaningful operational trade-offs between them, none of which is solved by simply picking one and self-hosting it [5]. SQS was chosen instead because it needs no broker to provision, patch, or scale manually at all, consistent with a Learner Lab account that cannot provision new IAM roles for a self-managed cluster in the first place, at the cost of the richer consumer-group semantics a system like Kafka offers.

Folding `mvs-processor` and `mvs-dashboard-api` into one Lambda was considered and rejected. One is driven by SQS batches arriving on the fog node's schedule; the other answers on-demand HTTP calls through API Gateway, with a distinct latency budget and no relationship to queue backlog. Sharing a single deployable would also share its reserved concurrency pool, so a spell of slow or erroring dashboard queries could eat into the capacity ingestion needs to keep pace with the queue—a dependency neither function should have on the other.

Hosting the dashboard directly on the EC2 instance, alongside the fog node, was considered and rejected. That would have exposed the dashboard on a raw EC2 public IP over a non-standard port, exactly the traffic shape campus and corporate WiFi networks commonly block by policy—a constraint discovered directly during this deployment, not assumed. Splitting the dashboard onto S3 plus API Gateway keeps the publicly demonstrated surface entirely on standard HTTPS ports and managed AWS domains.

Finally, whether Lambda could also take over the EC2-hosted fog node and sensors, leaving no always-on server anywhere in the system, was weighed and set aside for later, not ruled out entirely. Sensor sampling here runs as an unbroken loop reading and buffering on its own clock, not a bounded task with a natural start and end, so fitting it onto Lambda's invoke-and-terminate model would first require restructuring the sampling loop itself, and this project's cloud migration was scoped to the backend layer alone. That restructuring was judged too large to fit within the remaining project time, and is recorded instead as future work in Section V.

F. Security Considerations

Remote access to the EC2 instance is deliberately narrow: its security group opens exactly one inbound port, 8000, for the fog node's HTTP endpoint, and every other port, including 22 for SSH, stays closed. No key pair was attached at launch, so the only route into the instance for an operator is AWS Systems Manager Session Manager, which removes the standalone SSH attack surface entirely instead of merely restricting it. The single open port itself serves nothing beyond read-only, non-sensitive status data—never write access or a control-plane operation.

Both Lambda functions run under the AWS Academy Learner Lab account's shared LabRole instead of a role scoped to each function, because this account's service-control policies block a student from provisioning new IAM roles at all—there is no narrower option available inside this environment, not a preference for the broad one. Splitting that role apart is straightforward to specify even where it could not be implemented here: `mvs-processor` only ever needs two permissions, `sqs:ReceiveMessage/DeleteMessage` to drain the queue and `dynamodb:PutItem` to write the aggregate, while `mvs-dashboard-api` never writes at all and would be limited to read-oriented calls against DynamoDB, SQS, and Lambda metadata. Naming that gap explicitly instead of working around it quietly was a deliberate choice, not an oversight. Authentication was left off the dashboard's REST API for a related but separate reason: everything the endpoint returns is an aggregated, non-personal vessel reading, well short of anything access control would need to protect at this project's scale.

III. Implementation

A. Software Components and Libraries

The sensor, fog, processor, and dashboard-API components are all implemented in Python using Tornado for the two HTTP-facing services (the fog node and, locally, the dashboard) and boto3 for every AWS interaction. Trend charts on the dashboard are drawn in the browser with Chart.js, bundled into the static assets at build time instead of pulled from a content-delivery network at runtime, keeping the page free of any third-party network dependency. Docker Compose paired with LocalStack, an open-source AWS emulator, backs both everyday development and the CI pipeline, letting the same SQS, DynamoDB, and Lambda calls the production stack makes get rehearsed repeatedly without a real account, or its cost, anywhere in the loop.

The overall pipeline shape and several implementation-choice axes are knowingly adapted from earlier project work, not built entirely from scratch. This is a different category

of reuse from the brief's clause, which is phrased around a student's own earlier work: this adaptation draws on other prior codebases, not this student's prior submissions, so the practice is disclosed for transparency in full in `readme.txt`'s REUSE section rather than claimed as reuse authorized under that specific clause, including which specific axis (buffering strategy, alert-rule representation, routing shape, and so on) was made a genuinely distinct implementation choice, confirmed by reading the reused source directly, not assumed from memory.

B. Testing Strategy

Every module has an automated test suite written with pytest, exercised through hand-written fake AWS client objects and Tornado's AsyncHTTPTestCase driving a real bound socket rather than an in-process ASGI/WSGI shim, so the sensor and dashboard HTTP tests exercise the same networking path production traffic would. The fog-layer tests cover the aggregation arithmetic, the alert rules in Table I, and the `/ingest` endpoint's buffering behaviour end to end. The publisher tests cover the batching boundary explicitly: a 23-message window is asserted to produce batches of ten, ten, and three, not one call per message. The dashboard-Lambda tests cover every route in the flat dispatch table directly, including a scan-failure case where the pagination helper is mocked to raise and the response degrades to zero. The pagination fix itself is verified one layer below, in the data-access tests, with a four-page fake table (400, 400, 400, and 87 items) asserting the summed count is exactly 1287.

The sensor tests mock the dispatch executor and event loop instead of pausing on wall-clock sleeps, so all twelve assertions—bounded random-walk clamping at each profile's `lo`/`hi` limits, and the two independently configurable `SAMPLE_INTERVAL`/`DISPATCH_INTERVAL` knobs—run in well under a second. A script outside the pytest suite, `infra/burst.py`, complements it with a manual capacity check: it fires a configurable batch of synthetic messages, 2,000 by default spread across 32 worker threads, at the SQS queue using five `loadtest_`-prefixed sensor types kept apart from the five actual ones so burst traffic can never land in the DynamoDB partitions the live dashboard reads from, then polls the queue's `ApproximateNumberOfMessages` and `ApproximateNumberOfMessagesNotVisible` attributes until it drains.

C. Continuous Integration and Deployment

A GitHub Actions workflow runs the full automated test suite on every push to the repository, followed by a second job that brings the entire stack up with Docker Compose against LocalStack and runs a dedicated end-to-end verification script against the live containers. The stack is torn down afterwards regardless of outcome, so a failed run cannot leave a stale container running for the next one.

D. Live AWS Deployment and Defects Discovered

The complete system, including the sensor and fog tier, the SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend, was deployed to a live AWS account (an AWS Academy Learner Lab account) in region us-east-1, so the backend was deployed and tested on an actual public cloud platform, not only a local emulator. This deployment surfaced three defects, two before the system went live and one only after.

First, `mvs-dashboard-api`'s backend-stats endpoint reported the total number of items stored in DynamoDB through a single COUNT-select Scan call. A COUNT-select scan only counts the page it reads, typically capped around 1 MB of table data; a table larger than that returns a `LastEvaluatedKey` that must be followed with a further scan, or the reported count silently undercounts the table. The fix, a recursive version of `items_in_table()`, calls itself on `LastEvaluatedKey` until DynamoDB reports none left, summing each page's `Count` on the way back up the call stack—a distinct code shape from the do-while, while-true-with-array-push, and generator-plus-sum() implementations commonly used for the equivalent fix, deliberately not the same patch copied over unchanged.

Second, `fog/publisher.py` originally sent one SQS message per aggregate in a loop, issuing one outbound API call per aggregate even when several vessels' windows closed in the same flush cycle. The fix, `publish_batch()`, groups outgoing messages into batches capped at `SendMessageBatch`'s ten-entry ceiling, and the flush loop now makes a single call to it per window instead of repeating the old one-message-at-a-time publish call.

Third, and found only after the first two fixes were already verified live: the deployed dashboard's static assets all 404'd. `index.html` requests its CSS, JavaScript, and vendored Chart.js bundle at a `/static/` prefix, matching how the local Tornado server mounts them, but the initial S3 upload flattened every file to the bucket root instead of preserving that path. Every curl check against the JSON API had passed cleanly, because curl was only ever pointed at the API, never at the HTML page's asset references—the bug was found only once the live dashboard was opened in an actual browser and its network requests inspected directly, at which point three 404s were immediately visible. The fix re-uploaded `index.html` to the bucket root and everything else under the `static/` prefix it actually requests, re-verified by reloading the page and confirming every section populated with live, changing data. Fig. 2 shows the dashboard as it renders in a browser once every panel populates.

All three fixes were re-verified against the live deployment, not trusted from the unit test suite or from curl alone: the pagination and batching fixes were confirmed by unit tests asserting the exact page-summing and chunk-boundary behaviour, and the frontend fix was confirmed by loading the actual dashboard URL in a browser and reading its network requests directly, twice, fifteen seconds apart, watching the items-in-table count climb between polls. The source code, including the fixes described above, is available at [GitHub repository URL]; its `readme.txt` lays out local setup, the environment variables it expects, and a chronological account

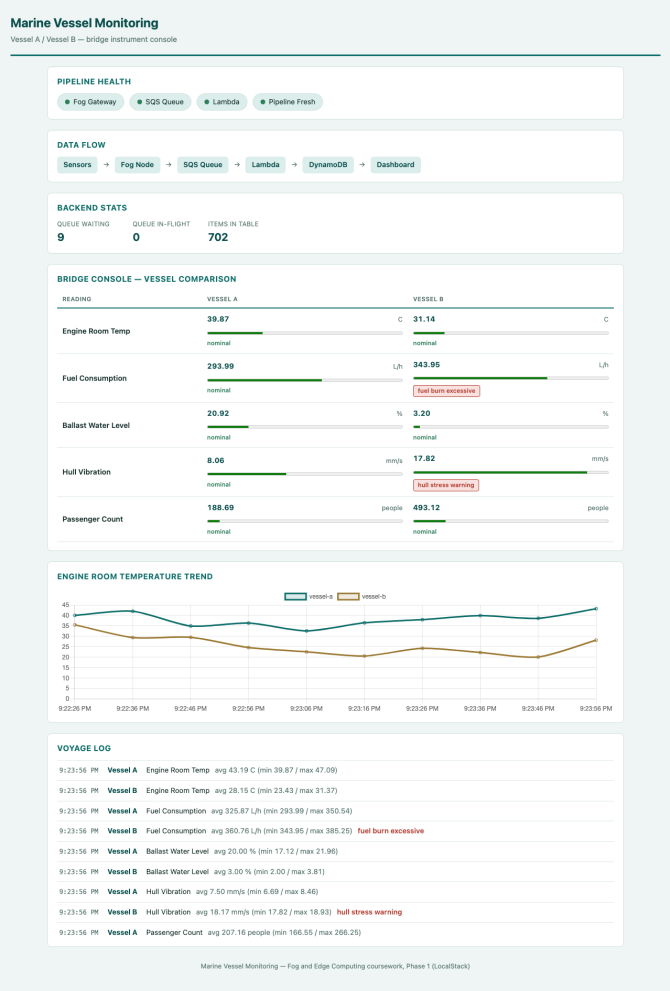

Fig. 2. The marine vessel monitoring dashboard rendered in a browser: pipeline health indicators, backend statistics, the Bridge Console side-by-side vessel comparison, an engine room temperature trend, and the voyage log with active alerts.

of the AWS rollout just described.

IV. Evaluation

A. Automated Test Coverage

Table II breaks the suite down by module as it stood at the last verified deployment, 120 tests in all. Every one passes against LocalStack, and the subset that actually builds AWS clients—where the pagination and batching fixes live—has additionally been re-run and confirmed against the live deployment covered in Section III-D.

B. Live Deployment Health

Polling the deployed API's `/api/health` endpoint independently confirmed every claimed dependency reachable: `gateway` (the fog node's health check), `queue`, `lambda`, and `pipeline` all reported true, with `freshest_age_seconds` under one second. That self-report was checked, not assumed: five AWS resources—the EC2 instance, both Lambda functions, the SQS queue, the DynamoDB table, and the API Gateway API—were each queried separately with the AWS CLI, and every one came back healthy independently. The EC2 instance reported state running against its bound Elastic IP with a security group allowing only port 8000 and no key pair, both Lambda functions reported state Active with their last update Successful, the event-source mapping between the queue and `mvs-processor` reported state Enabled, and the DynamoDB table reported status ACTIVE under on-demand billing.

The backend-stats items-in-table count was polled twice after the frontend fix, and grew from 59 to 374 to 425 across the course of verification, confirming a live, continuously updating pipeline, not static or cached data—the same live-reload check that caught the asset-path defect in Section III-D also served as positive evidence the fix worked, not just that the page returned a 200 status.

TABLE II
Automated Test Suite by Module

Module	Tests	Focus
sensors	12	bounded random walk, dual-rate config
fog (buffering+aggregation+alerts)	18	lock-free buffer, windowed aggregation, threshold rules
fog (publisher, HTTP)	22	SQS batching, socket-level /ingest tests
backend/processor	9	DynamoDB item shape, sort-key logic
backend/dashboard (HTTP + Lambda + data access)	38	routes, pagination fix, socket-level dashboard tests
thresholds proxy + validation	21	upstream fog reachability, payload validation

C. Cost Considerations

With the exception of the EC2 instance, every backend component operates inside AWS's request-based free-tier limits at the volume this project generates: DynamoDB's on-demand mode bills per request instead of provisioned capacity, both Lambda functions' call counts fall well short of the standing 1-million-request free allotment, and API Gateway and SQS follow the same per-request model, incurring nothing while idle. The EC2 instance is the exception—it hosts the fog node and sensor containers as persistent processes rather than short-lived invocations, so it is the sole resource billed continuously. That instance can, however, be powered down between demo sessions with no impact on the dashboard or its API: the only things that rely on it staying up are the `gateway` field returned by `/api/health` and the steady arrival of newly generated sensor readings.

D. Limitations

This evaluation carries four caveats worth naming outright, not left for a reader to infer. The Learner Lab account behind this deployment resets on a session basis, so it could not support the kind of sustained, multi-day uptime check a production system would need; everything reported here is confirmed behaviour within one continuous session, not a claim about

long-run reliability. That same account also blocks provisioning a second, narrower IAM role, so `mvs-processor` and `mvs-dashboard-api` both run under one broadly scoped LabRole instead of a role scoped to each function's needs—Section II-F covers this constraint in more detail, and it reflects an account limit, not a security choice. The third caveat is architectural: the fog and sensor tier still depends on a single EC2 instance, leaving that tier a single point of failure even though every AWS-managed component behind it is redundant; Section II-E's critical analysis explains why converting this tier to a serverless model was deferred, but deferring the fix does not close the availability gap it leaves open. The fourth caveat follows the same Learner Lab constraint as the first: `infra/burst.py`, the project's capacity-check script, was exercised against the LocalStack-backed queue during development, not against the deployed `mvs-vessel-agg` queue in the live account, so this report can describe how the script behaves but cannot state a measured sustained-throughput figure for the account under burst conditions.

V. Conclusion

By the time this project concluded, it had produced a working fog and edge computing pipeline for marine vessel condition monitoring: a configurable sensor and fog layer, a scalable cloud backend resting on a pair of separately deployed Lambda functions, and a deployment onto a public cloud account instead of a local stand-in. Equally central to the effort was holding each layer's design choices up against the alternatives that were available and being explicit about why one was chosen over another. All three layers cleared that bar—built, exercised under test, and running live on AWS.

If there is one lesson worth carrying out of this project, it is methodological rather than architectural, and it showed up twice, in two unrelated layers. The DynamoDB pagination bug sailed through the full local test suite and a complete LocalStack integration run without incident, and a build that stopped testing at the local-simulation stage would have shipped it none the wiser. The frontend asset-path defect slipped past every curl-based API check for much the same reason: curl only ever tests what it is aimed at, and nothing had been aimed at the page's asset references. Both were caught only once testing reached the real thing—an actual AWS account in the first case, an actual browser rendering the actual page in the second—and no quantity of further local testing would have surfaced either on its own.

Writing the application logic turned out to be the easy half of this project; working out the boundaries of what the Learner Lab account's platform and IAM rules would allow took longer, and so did learning that a green API check proves less than it looks like it proves about a page someone actually opens in a browser. Should those account restrictions ever be lifted, the sensor and fog tier is the obvious next target for a rewrite—off its one continuously running EC2 instance and onto a scheduled-Lambda, fully event-driven model—alongside splitting the shared LabRole into two purpose-built roles, one for `mvs-processor` and one for `mvs-dashboard-api`, each holding only the permissions it actually uses.

References

[1] G. Makridis, D. Kyriazis, and S. Plitsos, "Predictive maintenance leveraging machine learning for time-series forecasting in the maritime industry," in *Proc. 2020 IEEE 23rd Int. Conf. Intelligent Transportation Systems (ITSC)*, Rhodes, Greece, 2020, pp. 1–8.

[2] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Special Publication 800-145, Sep. 2011.

[3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12)*, Helsinki, Finland, Aug. 2012, pp. 13–16.

[4] B. M. Alencar, R. A. Rios, C. Santana, and C. Prazeres, "FoT-Stream: A Fog platform for data stream analytics in IoT," *Computer Communications*, vol. 164, pp. 77–87, 2020.

[5] R. Maharjan, M. S. H. Chy, M. A. Arju, and T. Cerny, "Benchmarking message queues," *Telecom*, vol. 4, no. 2, pp. 298–312, Jun. 2023.

[6] G. DeCandia *et al.*, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, Stevenson, WA, USA, Oct. 2007, pp. 205–220.

[7] M. Shahrad *et al.*, "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. 2020 USENIX Annu. Tech. Conf. (USENIX ATC '20)*, Jul. 2020, pp. 205–218.

[8] A. Akbulut and H. G. Perros, "Software Versioning with Microservices through the API Gateway Design Pattern," in *Proc. 2019 9th Int. Conf. Advanced Computer Information Technologies (ACIT)*, Ceske Budejovice, Czech Republic, Jun. 2019, pp. 289–292.